

Exercice GNN sur MNIST

Observations et gains incrémentaux

Wajd Ben Ismail

Stage LIG – Polytech Grenoble / ENSIMAG

26 juin 2026

Contexte

Cet exercice consiste à comparer trois architectures de Graph Neural Networks — GCN, GraphSAGE et GAT — sur le dataset MNIST, en transformant chaque image en graphe via une méthode de superpixels. L'objectif n'est pas d'atteindre le state of the art mais de vérifier empiriquement les différences théoriques entre les architectures étudiées en bibliographie.

1 Construction du graphe

1.1 Approche pixel par pixel (abandonnée)

La première approche consistait à représenter chaque pixel comme un nœud, avec des arêtes vers ses 8 voisins directs.

- **Nœuds** : 784 (un par pixel), feature = intensité du pixel
- **Arêtes** : ~ 4800 (connectivité 8)
- **Problème** : trop dense, trop lent, les pixels noirs (fond) polluent la représentation
- **Résultat GCN** : Accuracy $\sim 30\%$, F1 $\sim 23\%$

1.2 Approche superpixels (retenue)

Chaque image est segmentée en régions cohérentes via un algorithme de superpixels, réduisant drastiquement le nombre de nœuds.

- **Nœuds** : ~ 75 – 100 (un par superpixel)
- **Arêtes** : ~ 150 (régions voisines)
- **Avantage** : représentation plus compacte et sémantique

2 Évolution incrémentale du graph builder

2.1 Features des nœuds (9 au total)

Pour chaque superpixel, les features calculées sont : intensité moyenne, variance, intensité max, intensité min, position y normalisée, position x normalisée, taille relative, hauteur de la région, largeur de la région.

Modification	Détail	Accuracy	F1
SLIC, 3 features, 75 segments	intensité, x, y	30.40%	23.47%
SLIC, 5 features, 75 segments	+ variance, taille	31.95%	28.19%
SLIC, 9 features, 100 segments	+ max, min, h, w	33.90%	28.45%
SLIC, 9 features, compactness=0.05	suit mieux les contours	39.35%	34.83%
Quickshift, 9 features	kernel=2, dist=4, ratio=0.3	41.45%	39.10%

TABLE 1 – Impact des modifications du graph builder sur GCN (100 epochs)

2.2 Observations sur les algorithmes de superpixels

- **SLIC** : rapide, bon compromis, mais superpixels trop réguliers sur les chiffres complexes (5, 6, 8)
- **Quickshift** : suit mieux les contours, améliore significativement le chiffre 5 (F1 : 0.01 $\rightarrow$ 0.42) au prix d’une légère régression sur 7
- **Compactness** : paramètre clé de SLIC — 0.05 donne de meilleurs résultats que 1.0 (+4.5% F1)

3 Comparaison des architectures GNN

Tous les modèles utilisent le même graph builder (Quickshift, 9 features, 100 epochs) pour une comparaison équitable.

Modèle	Accuracy	F1 macro	Pooling
GCN	61.05%	59.36%	mean
GraphSAGE	84.80%	84.55%	mean
GraphSAGE	85.95%	85.64%	mean + max
GAT	86.00%	85.98%	mean + max
GATv2	—	—	à compléter

TABLE 2 – Comparaison des architectures GNN sur MNIST (100 epochs, 3 couches, Quickshift 9 features, sans filtrage du fond)

3.1 Observations GCN

- La moyenne simple des voisins perd trop d’information sur les chiffres à formes complexes
- Les chiffres simples (0, 1) sont bien classifiés (F1 $\sim$ 0.72–0.84)
- Les chiffres complexes (2, 5, 8) sont très mal classifiés (F1 $\sim$ 0.20–0.50)
- Cohérent avec la théorie : GCN ne discrimine pas l’importance des voisins

3.2 Observations GraphSAGE

- Gain de +25% en F1 par rapport à GCN
- 3 couches nettement meilleures que 2 couches (F1 : 77.53% $\rightarrow$ 84.55%)
- Toutes les classes atteignent un F1 > 0.75
- L’agrégation inductive capture mieux le contexte local des superpixels

4 Filtrage du fond

Une piste d’exploration proposée consiste à étudier l’impact d’un filtrage du fond *avant* la construction du graphe, comme première étape de réflexion sur ce problème.

4.1 Principe

Avant d’appliquer Quickshift, seuls les pixels appartenant au premier plan ($\text{intensité} > \text{seuil}$) sont conservés. Les superpixels obtenus sur fond uniquement sont supprimés, et le graphe est reconstruit avec un remapping des indices de nœuds.

Modèle	Sans filtrage	Avec filtrage
GCN	59.36%	60.64% (+1.3)
GraphSAGE (mean)	84.55%	84.20% (-0.35)
GAT (mean+max)	85.98%	85.53% (-0.45)

TABLE 3 – Impact du filtrage du fond sur le F1 macro, par modèle

4.2 Observation

Le filtrage du fond améliore légèrement GCN mais a un effet neutre ou légèrement négatif sur GraphSAGE et GAT. Une explication possible est que ces architectures plus expressives disposent déjà d’un mécanisme implicite (agrégation pondérée pour GraphSAGE, attention pour GAT) leur permettant d’apprendre à minimiser l’influence des nœuds de fond pendant l’entraînement. Le filtrage manuel n’apporterait alors pas d’information nouvelle, et pourrait même retirer un contexte spatial utile (la position du chiffre par rapport au fond).

4.3 Lien avec la différentiabilité

Ce filtrage reste lui-même non différentiable : le seuillage ($\text{intensité} > \text{seuil}$) est une fonction en escalier, de dérivée nulle presque partout. Il ne résout donc pas le problème de fond identifié dans RAGDNet (thinning, watershed et construction du graphe non différentiables), mais constitue une première expérimentation empirique avant d’explorer des formulations continues (ex. masquage par sigmoïde $\sigma((x - \text{seuil}) \times T)$, qui resterait dérivable).

5 Exploration d’une construction de graphe différentiable

En complément du filtrage du fond, une exploration plus poussée a été menée pour tester directement la piste évoquée par Alexandre et par la section *Discussion & Future Work* du papier RAGDNet : remplacer la construction de graphe heuristique (non différentiable) par une formulation apprise et différentiable.

5.1 Principe

Un module `DifferentiableGraphPooling` remplace Quickshift par une assignation *souple* des pixels à des clusters :

- Des centres de clusters sont des paramètres appris du modèle (et non plus calculés par un algorithme fixe)
- Chaque pixel reçoit une distribution de probabilités d’appartenance à chaque cluster, via $\text{assignment} = \text{softmax}(-\text{distance} \times T)$ au lieu d’une assignation dure (argmin)

- Les features de chaque cluster sont une moyenne pondérée des pixels, par leurs probabilités d'appartenance

Le pooling est intégré *dans* le modèle (et non plus en prétraitement), de sorte que le gradient de la loss de classification remonte jusqu'aux centres de clusters pendant l'entraînement.

5.2 Résultat

Approche	Accuracy	F1 macro
GCN + Quickshift (fixe, non différentiable)	61.05%	60.64%
GCN + Pooling différentiable (appris)	28.00%	21.74%

TABLE 4 – Comparaison construction de graphe fixe vs. différentiable (GCN, 1500 images train)

Le modèle différentiable échoue presque totalement sur les chiffres aux formes complexes (0, 3, 8 : F1 = 0.00), suggérant que le graphe complet (tous les clusters connectés, faute de k-NN différentiable simple) et le pooling global pondéré écrasent les détails géométriques nécessaires pour distinguer ces formes.

5.3 Interprétation

Ce résultat illustre concrètement le compromis discuté dans le papier RAGDNet entre pipeline heuristique et pipeline appris :

- Quickshift bénéficie de décennies d'optimisation pour la segmentation d'image et encode déjà un bon prior géométrique
- Le pooling différentiable, conçu et entraîné en quelques heures sans prior particulier, ne reconstruit pas encore ce niveau de qualité de segmentation
- Le gain théorique en différentiabilité (le gradient peut corriger la construction du graphe) ne se traduit pas automatiquement par un gain de performance : il faut une architecture de pooling suffisamment riche pour rivaliser avec l'heuristique qu'elle remplace

5.4 Pistes d'amélioration identifiées

À l'issue de cette première expérience (pooling couplé dès le départ, sans pré-entraînement), plusieurs pistes d'amélioration ont été identifiées et testées dans la section suivante :

- Remplacer le graphe complet par un k-NN différentiable entre clusters, plutôt qu'une connectivité totale
- Découpler l'entraînement du pooling (objectif propre) de celui du GCN/GAT final, pour une comparaison plus juste et interprétable
- Enrichir les features par pixel (gradients, contexte local) pour atteindre une richesse comparable à Quickshift (9 features)

6 Approche découplée : pooling pré-entraîné puis figé

Contrairement à l'expérience précédente où le pooling apprenait *en même temps* que le GCN final (sans guidage propre), cette seconde approche sépare les deux étapes :

1. Le pooling est entraîné **seul**, avec un objectif de reconstruction : chaque pixel doit pouvoir être approximé à partir des features pondérées de son cluster ($\mathcal{L} = MSE(reconstruction, features)$)
2. Une fois entraîné, le pooling est **figé** (`requires_grad = False`)

- Il sert alors de remplaçant direct à Quickshift pour construire les graphes, sans aucune modification du GCN/GAT déjà utilisé (architecture strictement identique)

Les features par pixel sont également enrichies à 9 dimensions (intensité, moyenne locale, max/min locaux, position, gradients de Sobel) afin de rester comparables à celles utilisées avec Quickshift. Le graphe de clusters est construit par k plus proches voisins (k -NN) sur les positions des clusters actifs, plutôt qu'un graphe complet.

6.1 Résultats

Modèle	Construction du graphe	Accuracy	F1 macro
GCN	Quickshift (fixe)	60.64%	60.64%
GCN	Pooling différentiable figé ($k=4$)	76.70%	76.06%
GAT	Quickshift (fixe)	86.00%	85.98%
GAT	Pooling différentiable figé ($k=4$)	84.45%	84.21%
GAT	Pooling différentiable figé ($k=2$)	81.85%	81.31%

TABLE 5 – Comparaison Quickshift vs. pooling différentiable figé, à architecture GCN/GAT strictement identique

6.2 Observations

- Pour GCN, le pooling différentiable figé **dépasse nettement** Quickshift (+15,4 points de F1). Le découplage (objectif de reconstruction propre + features enrichies) change radicalement la conclusion par rapport à la première tentative couplée (21,74% de F1)
- Pour GAT, le pooling différentiable reste légèrement **en dessous** de Quickshift (-1,8 point de F1), mais l'écart est faible
- Diagnostic de densité du graphe : Quickshift produit en moyenne 15,8 nœuds et 3,11 arêtes/nœud, contre 29,6 nœuds et 8,00 arêtes/nœud pour le pooling différentiable ($k=4$). Réduire k à 2 (4,00 arêtes/nœud, plus proche de Quickshift) **dégrade** le résultat (81,31% au lieu de 84,21%) : l'hypothèse selon laquelle la sur-densité du graphe pénaliserait l'attention de GAT est donc **réfutée** empiriquement : GAT bénéficie au contraire de plus de connectivité, pas moins
- Interprétation : GAT dispose déjà d'un mécanisme (l'attention) pour pondérer l'importance de ses voisins et compense ainsi en partie les imperfections d'un graphe construit par une heuristique mature (Quickshift). GCN, plus simple (moyenne non pondérée), profite davantage d'un graphe mieux construit en amont

6.3 Piste en cours : fine-tuning conjoint

Une piste supplémentaire est explorée pour tenter de dépasser Quickshift également pour GAT : au lieu de figer le pooling après son pré-entraînement par reconstruction, on le **dégèle** et on l'entraîne *conjointement* avec GAT sur la tâche de classification finale. L'hypothèse est que le gradient de classification peut affiner les centres de clusters spécifiquement pour la tâche, au-delà de ce que permet le seul objectif de reconstruction. Expérience en cours sur GPU (A100, GRICAD/Bigfoot) – résultats à compléter.

7 Pistes explorées en parallèle

En complément du fine-tuning conjoint, trois pistes additionnelles sont explorées pour mieux cerner où se situe le levier de performance le plus prometteur.

7.1 Piste A – Réduction de résolution avant construction du graphe

Hypothèse : réduire la résolution de l'image ($28 \times 28 \rightarrow 14 \times 14$, par interpolation avec anti-aliasing) avant le filtrage du fond et Quickshift pourrait concentrer le signal utile et réduire le bruit de segmentation, à architecture GAT inchangée. Implémentation : `graph_builder_downsampled.py`, réutilisant les mêmes 9 features et le même filtrage de fond déjà validés, appliqués sur l'image réduite. Résultats à compléter.

7.2 Piste B – SLIC différentiable

SLIC (Simple Linear Iterative Clustering), méthode de référence pour les superpixels, repose sur des itérations dures (assignation par distance minimale). Une formulation différentiable de SLIC existe dans la littérature (*Superpixel Sampling Networks*, Jampani et al., 2018, présent dans la bibliographie initiale du stage). Le module `DifferentiableGraphPooling` déjà développé en est une version simplifiée (assignation souple par softmax sur la distance aux centres de clusters, sans la composante itérative de SLIC). Piste de prolongement : renforcer la contrainte de proximité spatiale dans l'assignation, à l'image de SLIC qui combine couleur et position.

7.3 Piste C – Watershed différentiable

Le watershed utilisé dans RAGDNet (et dans Deep Watershed Transform, Bai & Urtasun, 2017, également présent dans la bibliographie initiale) reste un algorithme discret non différentiable. L'approche de Deep Watershed Transform ne rend pas le watershed lui-même différentiable, mais fait apprendre par un réseau la carte d'énergie qui *alimente* le watershed : le gradient peut ainsi remonter jusqu'à cette carte d'énergie, sans traverser l'opération de watershed elle-même. Piste de prolongement : étudier si une approche analogue est transposable au pipeline RAGDNet (apprentissage de la carte de priorité utilisée pour l'expansion des branches du squelette, en amont du watershed marker-controlled).

8 Watershed différentiable end-to-end avec GAT

8.1 Principe

Contrairement à l'approche découplée (pooling pré-entraîné puis figé), cette piste rend le Watershed lui-même différentiable et l'intègre directement *dans* le modèle, entraîné conjointement avec GAT dès le départ.

- Un CNN léger (2 convolutions) apprend une carte d'assignation *souple* des pixels à n zones (bassins versants), via un softmax sur les canaux de sortie
- Les features de chaque nœud (intensité, position x, y) sont obtenues par moyenne pondérée des pixels selon cette carte d'assignation
- L'adjacence entre nœuds est calculée par produit matriciel de la carte d'assignation avec elle-même (deux zones sont connectées si elles partagent des pixels proches), puis un top-k est appliqué pour ne garder que les arêtes les plus fortes
- Le graphe résultant (nœuds + arêtes) est reconstruit à la volée à chaque forward pass et passé directement à GAT

Le gradient de la loss de classification remonte ainsi jusqu'au CNN de segmentation, sans étape de gel/dégel comme dans l'approche découplée.

Modèle	Accuracy	F1 macro
GAT + Quickshift (fixe)	86.00%	85.98%
GAT + Pooling différentiable figé (découplé)	84.45%	84.21%
GAT + Watershed différentiable end-to-end	92.80%	92.64%

TABLE 6 – Comparaison des approches de construction de graphe pour GAT

8.2 Résultats

8.3 Observations

- Gain de +6,66 points de F1 par rapport à Quickshift, et de +8,43 points par rapport à l’approche découplée précédente
- Toutes les classes atteignent un $F1 > 0,89$, sans faiblesse marquée sur les chiffres complexes (contrairement à GCN)
- Contrairement à l’approche découplée (pré-entraînement par reconstruction puis gel), ici la segmentation est guidée *dès le départ* par le signal de classification, ce qui semble permettre d’apprendre des zones plus discriminantes pour la tâche finale
- Confirme l’hypothèse de la section précédente : le frein n’était pas la différentiabilité en soi, mais la richesse de l’architecture de pooling (ici un CNN spatial, contre une simple distance aux centres de clusters)

9 Binarisation différentiable pour la suppression du fond

9.1 Principe

En complément du Watershed différentiable, une étape de suppression du fond est intégrée *avant* le calcul de la carte d’assignation, de façon différentiable : le seuillage dur évoqué en section 4.3 ($intensit > seuil$) est remplacé par une sigmoïde à fort gain :

$$masque(x) = \sigma(G \times (x - \tau))$$

avec G un gain élevé (forçant une transition quasi-binaire) et τ un seuil. La carte d’assignation du Watershed est multipliée par ce masque, ce qui annule progressivement la contribution des pixels de fond dans la construction des nœuds, sans introduire de discontinuité dans le gradient.

9.2 Résultats

Modèle	Accuracy	F1 macro
GAT + Watershed différentiable (sans masque)	92.80%	92.64%
GAT + Watershed différentiable + Masque soft	93.10%	92.93%

TABLE 7 – Impact de la suppression différentiable du fond

9.3 Observations

- Gain modeste mais consistant de +0,29 point de F1
- Contrairement au filtrage dur testé en section 4 (neutre voire négatif sur GAT), la version différentiable du masquage est ici légèrement positive : le seuil τ et le gain G étant appris conjointement avec le reste du modèle, le masque peut s’ajuster pour ne retirer que le bruit réellement non informatif, plutôt qu’un seuil fixe choisi a priori

- Confirme que rendre une étape différentiable (même simple, comme un seuillage) permet de l'intégrer proprement dans l'optimisation globale, sans la régression observée lors du filtrage non différentiable

10 À compléter